

<https://www.kaggle.com/code/nadezhda2019/generation-diff/> +

Различают следующие сети для генерации:

VAE, Диффузионные (SOTA), Авторегрессионные, GAN

1. Используйте Библиотеку diffusers от Hugging Face для генерации изображения.



2. Генеративно-состязательные сети.

<https://www.kaggle.com/code/nadezhda2019/introduction-to-gans>

- 3.

Проект. Дообучение диффузионной модели





Представьте, что вы хотите, чтобы диффузионная модель могла генерировать фотографии с вами, вашей кошкой или чебурашкой. Но изначально модель не понимает, как выглядите вы, ваша кошка и чебурашка. Нужно, чтобы модель выучила это, то есть дообучить её на ранее неизвестный ей концепт. Это распространённая задача в компьютерном зрении.

Начнём с чебурашки. Если у вас останется время, то можете проделать такое же дообучение с другими объектами.

Краткое описание

План проекта. Вам нужно:

1. Показать, что диффузионная модель не знает, что такое чебурашка.
2. Дообучить LoRA на диффузионную модель на датасете картинок с чебурашкой.
3. Показать, что с выученной LoRA диффузионная модель может генерировать чебурашку.

Датасет состоит всего из трёх изображений. Это мало, но такого количества хватает для дообучения LoRA. Скачайте изображения: [одно](#), [второе](#) и [третье](#).

Задание

Выполните проект по этапам.

Этап 1. Работа с данными и демонстрация работы сырой модели

1. **Работа с данными**

- Загрузите картинки и визуализируйте их. У вас должны получиться три изображения с чебурашкой из фильма.
- Реализуйте свой класс датасета. Он должен загружать картинки, ресайзить их к разрешению 512×512 пикселей и нормировать так, чтобы среднее и std были равны 0.5.

2. Работа с оригинальной моделью

- Создайте DiffusionPipeline из библиотеки diffusers. Используйте модель runwayml/stable-diffusion-v1-5. Это не лучшая модель, но она легковесная и её качество достаточно для учебного проекта.
- Сгенерируйте с помощью созданного пайплайна изображение по запросу "<cheburashka> with the Eiffel Tower in the background". Покажите, что модель не знает, что такое <cheburashka>.
- Посмотрите в функцию [encode_prompt](#) в StableDiffusionPipeline и закодируйте с помощью неё текст "<cheburashka> plushie". Полученные текстовые эмбединги сохраните, они понадобятся при обучении модели.

Если при работе с документацией вам не всё будет понятно, изучите пример кода под катом.

Пример кода для encode_prompt

```
(
    prompt_embeds,
    negative_prompt_embeds
) = pipe.encode_prompt(instance_prompt, "cuda", num_images_per_prompt=1,
do_classifier_free_guidance=False)
```

- Удалите пайплайн и почистите кеш памяти, чтобы не занимать память:
- `import gc`
- `del pipe # удаляем пайплайн`
- `gc.collect() # чистим память`

```
torch.cuda.empty_cache() # чистим кеш cuda
```

Результаты этапа: отображены картинки, на которых модель будет обучаться; показано, что оригинальная модель не знает наш концепт; написан класс датасета; закодирован текстовый промпт.

Этап 2. Дообучение модели

1. Загрузка моделей

- Загрузите из библиотеки diffusers шедулер DDPM Scheduler, основную модель UNet2DConditionModel и VAE AutoencoderKL для модели runwayml/stable-diffusion-v1-5.
- Зафиксируйте веса основной модели и VAE с помощью `x.requires_grad_(False)`.

2. Добавление LoRA

Добавьте к модели LoRA. Используйте LoraConfig из библиотеки peft и метод add_adapter основной модели. В конфиге для LoRA используйте init_lora_weights="gaussian" и target_modules=["to_k", "to_q", "to_v", "to_out.0"].

Здесь:

- init_lora_weights — это параметр, который указывает, как надо инициализировать веса для LoRA.
- target_modules — это параметр, который отвечает, на какие модули модели нужно навесить LoRA.

Ранг LoRA можете использовать 128 или попробовать другие.

3. Реализация пайплайна дообучения

4. Реализуйте цикл обучения модели со стандартной функцией потерь. При необходимости изучите [пример реализации обучения](#).
5. Опишите каждый этап обучения. Можно сделать это в markdown ячейках или в комментариях в коде. Опишите:
 - зачем нужен vae, что он делает в пайплайне;
 - как семплируется диффузионное время;
 - как зашумляется изображение;
 - что идёт на вход основной модели;
 - какой лосс считается;
 - какие дополнения нужно сделать для этой модели при подсчёте лосса.
6. Можете использовать такие параметры обучения:
 - learning_rate=2.0e-05,
 - max_train_steps=1000,
 - train_batch_size=1,
 - max_grad_norm=1.0,
 - lr_scheduler="constant",
 - lr_warmup_steps=0,
 - snr_gamma=5.0,
 - lora_rank=128.

Можете попробовать поварьировать параметры.

Если всё сделано верно, 1000 шагов обучения будут идти 15-20 минут.

7. Сделайте checkpoint-ы модели (каждые 500 шагов или чаще).
8. Дообучите модель и сохраните полученные чекпоинты.

Результаты этапа: написан пайплайн обучения модели; в комментариях описаны основные этапы обучения; модели инициализированы, и к основной модели добавлена LoRA; модель обучена, сохранены её чекпоинты.

Этап 3. Демонстрация результатов

1. Генерация изображений с обученной LoRA

- Создайте снова пайплайн генерации.
- В unet созданного пайплайна добавьте LoRA и загрузите обученный чекпоинт.
- Сгенерируйте изображения по запросам "<cheburashka> with the Eiffel Tower in the background", "<cheburashka> plushie", "<cheburashka> in sketch style", "<cheburashka> riding a bicycle". Тем самым покажите, что модель теперь знает, что такое чебурашка. Попробуйте разные сохраненные чекпоинты и выберите лучший.

Результат: сгенерированы изображения чебурашки по заданным промптам; показано, что теперь модель может генерировать чебурашку.